Experiment No. 5
Apply Genetic Algorithm for Real-World Problem (Travelling Sales Person)
Date of Performance:
Date of Submission:

Aim: To apply the Genetic Algorithm (GA) to solve a real-world optimization problem.

Theory:

Genetic Algorithms (GAs) mimic the process of natural selection to solve optimization problems. They are particularly useful for discrete, non-linear, or non-convex problems.

Steps:
1. Initialization: Generate an initial population of candidate solutions (chromosomes).
2. Selection: Choose parents based on their fitness.
3. Crossover: Combine genes of parents to create offspring.
4. Mutation: Randomly alter genes in offspring to maintain diversity.
5. Termination: Stop when a solution meets the desired fitness level or after a set number of generations.
6. Applications: Scheduling, routing, feature selection in machine learning.

Procedure/Algorithm:

a. Define the optimization problem and represent it as a chromosome.
b. Initialize a population randomly.
c. Evaluate the fitness of each individual in the population.
d. Repeat for a specified number of generations:
    i. Select individuals based on fitness.
    ii. Perform crossover to generate offspring.
    iii. Apply mutation to introduce diversity.
    iv. Replace the old population with the new one.
e. Display the best solution.

Code:

```
import numpy as np
import random
import matplotlib.pyplot as plt
import pandas as pd
cities = {
```

```
    'A': (0, 0),
    'B': (10, 20),
    'C': (20, 15),
    'D': (30, 10),
    'E': (25, 25)
}
cities_list = list(cities.keys())
num_cities = len(cities_list)
def route_distance(route):
    dist = 0
    for i in range(len(route) - 1):
        city1, city2 = cities[route[i]], cities[route[i + 1]]
        dist += np.linalg.norm(np.array(city1) - np.array(city2))
    # Return to the starting city
    dist += np.linalg.norm(np.array(cities[route[-1]]) - np.array(cities[route[0]]))
    return dist
def initial_population(pop_size):
    return [random.sample(cities_list, num_cities) for _ in range(pop_size)]
def tournament_selection(population, k=3):
    selected = random.sample(population, k)
    return min(selected, key=route_distance)
def ordered_crossover(parent1, parent2):
    size = len(parent1)
    start, end = sorted(random.sample(range(size), 2))
    child = [None] * size
    child[start:end + 1] = parent1[start:end + 1]
    remaining = [city for city in parent2 if city not in child]
```

```
    j = 0
    for i in range(size):
        if child[i] is None:
            child[i] = remaining[j]
            j += 1
    return child
def mutate(route, mutation_rate=0.1):
    if random.random() < mutation_rate:
        i, j = random.sample(range(len(route)), 2)
        route[i], route[j] = route[j], route[i]
    return route
def genetic_algorithm(pop_size=5, generations=3, mutation_rate=0.1):
    population = initial_population(pop_size)
    history = []
    for gen in range(generations):
        print(f"\nGeneration {gen+1}:")
        print("Initial Population:")
        for individual in population:
            print(individual, "-> Distance:", route_distance(individual))
        new_population = []
        for _ in range(pop_size):
            parent1 = tournament_selection(population)
            parent2 = tournament_selection(population)
            print(f"Selected Parents: {parent1}, {parent2}")
            child = ordered_crossover(parent1, parent2)
            print(f"Child Before Mutation: {child}")
            child = mutate(child, mutation_rate)
```

```
            print(f"Child After Mutation: {child}")
            new_population.append(child)
        population = new_population
        best_route = min(population, key=route_distance)
        best_distance = route_distance(best_route)
        history.append((gen + 1, best_route, best_distance))
        print(f"Best Route: {best_route} -> Best Distance: {best_distance:.2f}")
    return best_route, best_distance, history
best_route, best_distance, history = genetic_algorithm()
df_history = pd.DataFrame(history, columns=['Generation', 'Best Route', 'Best Distance'])
print("\nEvolution History:")
print(df_history)
plt.figure(figsize=(8, 6))
for i in range(len(best_route)):
    city1, city2 = cities[best_route[i]], cities[best_route[(i + 1) % len(best_route)]]
    plt.plot([city1[0], city2[0]], [city1[1], city2[1]], 'bo-')
plt.scatter([cities[c][0] for c in cities], [cities[c][1] for c in cities], c='red', marker='o')
for city, coord in cities.items():
    plt.text(coord[0], coord[1], city, fontsize=12, ha='right')
plt.title(f'Traveling Salesman Best Route (Distance: {best_distance:.2f})')
plt.xlabel('X Coordinate')
plt.ylabel('Y Coordinate')
plt.show()
df_best_route = pd.DataFrame({'City': best_route, 'X Coordinate': [cities[c][0] for c in best_route],
                              'Y Coordinate': [cities[c][1] for c in best_route]})
print("\nBest Route DataFrame:")
print(df_best_route)
```

Output:

Generation 1:
Initial Population:
['A', 'C', 'D', 'B', 'E'] -> Distance: 109.70774702266611
['D', 'E', 'A', 'C', 'B'] -> Distance: 109.70774702266613
['C', 'E', 'B', 'D', 'A'] -> Distance: 105.97518456502253
['D', 'E', 'B', 'A', 'C'] -> Distance: 90.16379626418063
['D', 'C', 'A', 'B', 'E'] -> Distance: 90.16379626418063
Selected Parents: ['D', 'E', 'B', 'A', 'C'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'E', 'B', 'A', 'C']
Child After Mutation: ['D', 'E', 'B', 'A', 'C']
Selected Parents: ['D', 'C', 'A', 'B', 'E'], ['D', 'E', 'B', 'A', 'C']
Child Before Mutation: ['D', 'C', 'A', 'B', 'E']
Child After Mutation: ['D', 'C', 'A', 'B', 'E']
Selected Parents: ['D', 'C', 'A', 'B', 'E'], ['D', 'E', 'B', 'A', 'C']
Child Before Mutation: ['D', 'C', 'A', 'B', 'E']
Child After Mutation: ['D', 'C', 'A', 'B', 'E']
Selected Parents: ['D', 'E', 'B', 'A', 'C'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'E', 'B', 'A', 'C']
Child After Mutation: ['D', 'E', 'B', 'A', 'C']
Selected Parents: ['D', 'C', 'A', 'B', 'E'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'C', 'A', 'B', 'E']
Child After Mutation: ['D', 'C', 'A', 'B', 'E']
Best Route: ['D', 'E', 'B', 'A', 'C'] -> Best Distance: 90.16

Generation 2:
Initial Population:
['D', 'E', 'B', 'A', 'C'] -> Distance: 90.16379626418063
['D', 'C', 'A', 'B', 'E'] -> Distance: 90.16379626418063
['D', 'C', 'A', 'B', 'E'] -> Distance: 90.16379626418063
['D', 'E', 'B', 'A', 'C'] -> Distance: 90.16379626418063
['D', 'C', 'A', 'B', 'E'] -> Distance: 90.16379626418063
Selected Parents: ['D', 'E', 'B', 'A', 'C'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'E', 'B', 'A', 'C']
Child After Mutation: ['D', 'E', 'B', 'A', 'C']
Selected Parents: ['D', 'E', 'B', 'A', 'C'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'E', 'C', 'A', 'B']
Child After Mutation: ['D', 'E', 'C', 'A', 'B']
Selected Parents: ['D', 'E', 'B', 'A', 'C'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'B', 'E', 'A', 'C']
Child After Mutation: ['D', 'B', 'E', 'A', 'C']
Selected Parents: ['D', 'C', 'A', 'B', 'E'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'C', 'A', 'B', 'E']

Child After Mutation: ['D', 'C', 'A', 'B', 'E']
Selected Parents: ['D', 'C', 'A', 'B', 'E'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'C', 'A', 'B', 'E']
Child After Mutation: ['D', 'C', 'A', 'B', 'E']
Best Route: ['D', 'E', 'B', 'A', 'C'] -> Best Distance: 90.16

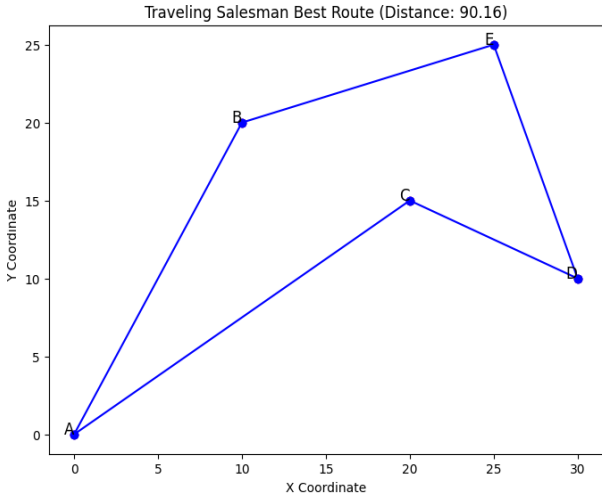
Generation 3:
Initial Population:
['D', 'E', 'B', 'A', 'C'] -> Distance: 90.16379626418063
['D', 'E', 'C', 'A', 'B'] -> Distance: 96.71308773833664
['D', 'B', 'E', 'A', 'C'] -> Distance: 109.70774702266613
['D', 'C', 'A', 'B', 'E'] -> Distance: 90.16379626418063
['D', 'C', 'A', 'B', 'E'] -> Distance: 90.16379626418063
Selected Parents: ['D', 'C', 'A', 'B', 'E'], ['D', 'E', 'B', 'A', 'C']
Child Before Mutation: ['D', 'C', 'A', 'B', 'E']
Child After Mutation: ['D', 'C', 'A', 'B', 'E']
Selected Parents: ['D', 'E', 'B', 'A', 'C'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'E', 'B', 'A', 'C']
Child After Mutation: ['D', 'E', 'B', 'A', 'C']
Selected Parents: ['D', 'C', 'A', 'B', 'E'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'C', 'A', 'B', 'E']
Child After Mutation: ['D', 'C', 'A', 'B', 'E']
Selected Parents: ['D', 'C', 'A', 'B', 'E'], ['D', 'E', 'B', 'A', 'C']
Child Before Mutation: ['D', 'C', 'A', 'E', 'B']
Child After Mutation: ['D', 'C', 'A', 'E', 'B']
Selected Parents: ['D', 'C', 'A', 'B', 'E'], ['D', 'C', 'A', 'B', 'E']
Child Before Mutation: ['D', 'C', 'A', 'B', 'E']
Child After Mutation: ['D', 'C', 'A', 'B', 'E']
Best Route: ['D', 'C', 'A', 'B', 'E'] -> Best Distance: 90.16

Evolution History:

	Generation	Best Route	Best Distance
0	1	[D, E, B, A, C]	90.163796
1	2	[D, E, B, A, C]	90.163796
2	3	[D, C, A, B, E]	90.163796

Best Route DataFrame:

	City	X Coordinate	Y Coordinate
0	D	30	10
1	C	20	15
2	A	0	0
3	B	10	20
4	E	25	25



Conclusion: The Genetic Algorithm efficiently finds an optimal or near-optimal travel route by evolving solutions over multiple generations

Answer the following questions:

1. What is the role of mutation in GA?

Mutation in a Genetic Algorithm (GA) helps introduce variety in the population by making small random changes in a solution (chromosome). This prevents the algorithm from getting stuck in local optima (a solution that seems best but isn't the overall best). Mutation works by swapping, changing, or flipping certain parts of a solution, ensuring that new possibilities are explored. Without mutation, GA might only keep improving existing solutions without discovering better ones. It plays a key role in maintaining diversity, helping the algorithm find the best solution in a more efficient way

2. How do selection mechanisms influence GA performance?

Selection mechanisms in a Genetic Algorithm (GA) determine which individuals (solutions) are chosen to create the next generation. They influence GA performance by balancing exploration (searching new areas) and exploitation (refining good solutions). Common selection methods include roulette wheel selection (probability based on fitness), tournament selection (competing among random individuals), and rank selection (ranking individuals and selecting based on position). If selection pressure is too high (always picking the best solutions), GA may converge too early to a suboptimal solution. If it's too low (random selection), GA may take too long to find a good solution. A well-chosen selection method ensures a good mix of strong solutions while maintaining diversity for better optimization.